Temporally Stable Credit Risk Estimation

Rohit Pujar Nagraj Shreyas Nagesh Xiaolin Zhou Yiyang Duan

Abstract

This report details our participation in an ongoing Kaggle competition hosted by Home Credit, aimed at predicting loan default risks among individuals with minimal or no credit history. The challenge emphasized the need for models that maintain predictive accuracy over time. We adopted an ensemble approach using LightGBM, XGBoost, and CatBoost, enhanced by a voting mechanism for improved stability and accuracy. Our preprocessing strategy involved feature aggregation and transformation, and our final model achieved an area under the receiver operating characteristic curve (AUC) of 0.78. Evaluated on a gini stability metric, our model scored 0.58, demonstrating its efficacy in maintaining performance over time and underscoring the potential of machine learning in responsible lending and financial inclusion. The code is published in a Github repository.

1. Introduction

This project is part of an ongoing Kaggle competition organized by Home Credit, focused on predicting loan defaults among clients with little to no credit history. Recognizing the importance of stability in credit risk models, this competition evaluates the robustness of predictive algorithms over time. Our challenge is not only to assess risk accurately but also to ensure our models adapt to evolving economic and borrower conditions without losing predictive accuracy.

The dataset consists of a hierarchical structure of files, the hierarchy going two levels deep. When merged, we obtain data in a tabular format with 1.5M rows and 1035 columns. Figure 1 depicts the information contained in the dataset.

The Home Credit group had conducted a similar Kaggle competition in 2018 (et al., 2018) where the aim was to purely predict credit risk. It did not have the temporal stability aspect to it. On the other hand, the current competition is heavily focused on temporal stability which also interests us as a team for the reasons that it has another aspect to the modelling over just the classical prediction task.

When looking through the submissions for the competition

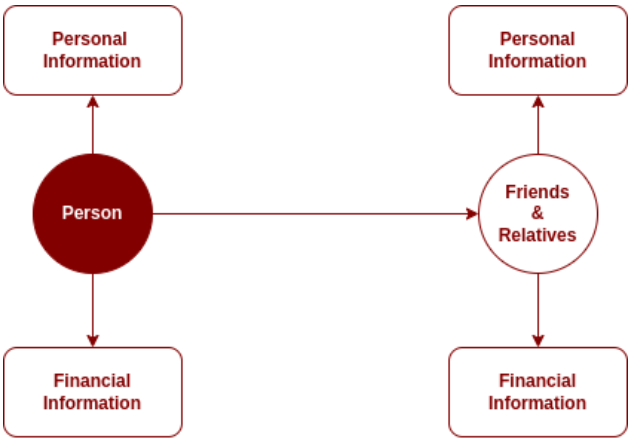


Figure 1. Information contained in the dataset

from 2018, we found (Narayanan, 2021a) to be a particularly detailed EDA approach. Our feature engineering techniques were a direct result of this EDA since the data provided for both the competitions were similar. This not only helped us expedite the feature engineering process but also, guided us to look at the most useful subset of data without having to go through all the 1000+ columns. (Narayanan, 2021b) implements cross validation with LightGBM for the 2018 competition and we used this approach as a starting point. (Rutecki, 2023) also helped us concrete our plans of bagging multiple classifiers as its a well known technique to improve performance.

The codebase is published in a Github repository.[1]

2. Methods

Due to the massive size of the data, extracting the important information is pivotal work to refine our model. Therefore, we divide the task two primary tasks: Feature Engineering and Modelling, the flow of which is depicted on Figure 2.

Feature Engineering

- **Removing sparse columns:** We remove any features that have more than 70% null values, since this feature can only provide limited information to prediction and

[1] https://github.com/ShreyasN01/CreditRiskEstimation

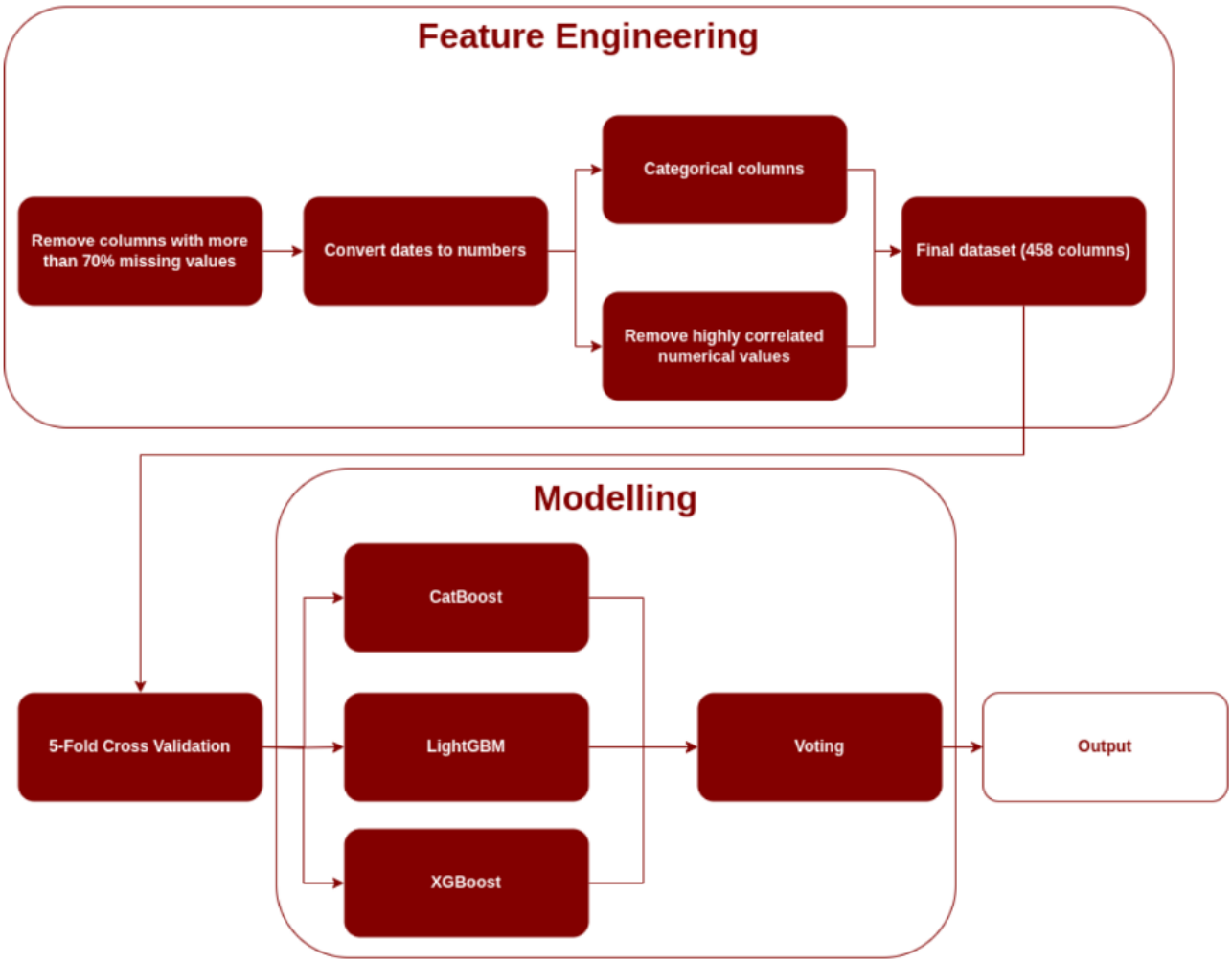


Figure 2. Model Framework

sometimes can skew the performance of the model.

- **Handling dates:** We observe that a lot of columns contain date values as strings, which are meaningless in the present format to the model. Thus, we convert these date columns into more meaningful features such as number of days or duration.

- **Categorical data:** The models we use in our final approach can intrinsically handle categorical columns and thus, we do not pre-process the categorical columns in our final approach. However, when we experimented with models that cannot handle categorical columns, we used label encoding, one-hot encoding and bucketing to convert the categories to numeric values.

- **Correlation reduction:** If two columns have a high correlation, this means that choosing either one of the two columns is enough to construct the model, not only reducing the computation, but also decreasing the probability of overfitting.

After pre-processing the data, we end up with 458 features train the model further.

Modelling

- **Cross-validation:** 5-fold cross-validations are applied and each fold includes 458 columns, which not only increase the generalizability of the model and mitigate the risk of overfitting, but also provide a rigorous framework, enabling the stability and reliability of the model across various subsets of data.

- **Model selection:** For our final approach, we train three boosting models, **CatBoost**, **LightGBM**, and **XGBoost**. We have two reasons we pick these models. The first being these models can implicitly handle cate-

case_id	score
57543	0.0108652022865331
57549	0.0546488954736736
57551	0.00741195179321866
57552	0.00928447636162376
57569	0.0661679629547249

Figure 3. Sample submission

gorical features and the second being boosting mechanisms are still the state-of-the-art for tabular data.

- **Ensemble technique:** A soft voting mechanism is employed as an ensemble method. We take the prediction probability from each model, sum it up and then average it across the models to get our final prediction. This helps us achieve two things, the first being that we now have continuous outputs which we can treat as regression outputs (which is what the competition requires), and the second is that the ensemble significantly improves our performance as discussed in the Results section. Figure 3 depicts the sample final output our model produces that is treated as a submission to Kaggle.

Evaluation Metrics

Area Under the ROC Curve, AUC, is applied to evaluate the performance of the model locally, which is effective in assessing the ability of the model to distinguish among various classes. However, we can't effectively test the weekly deviation as it might not represent the test set.

3. Results

The Kaggle competition calculates a gini stability metric as follows: First, a gini score is calculated for each *WEEK_NUM* given by:

$$gini = 2 * AUC - 1$$

Then, a linear regression *a.x + b* is fit through the weekly gini scores, and a *falling_rate* is calculated as *min(0, a)*.

Finally, the variability of the predictions are calculated by taking the standard deviation of the residuals from the above linear regression, applying a penalty to model variability.

The final metric is calculated as shown in Eq. (1).

Table 1. Performance of models experimented

Model	Handled date and NULL values	Proposed FE pipeline	Score
Random Forests	0.55	0.58	0.68
Logistic Regression	0.54	0.56	0.69
SVM	0.55	0.56	0.7
XGBoost	0.6	0.64	0.74
CatBoost	0.61	0.65	0.75
LightGBM	0.6	0.62	0.73
Ensemble model of XGB, CatBoost, LightGBM	0.73	0.74	0.78

Further, we score our models locally using the *AUC* score as the variability between models is not a metric we have control over.

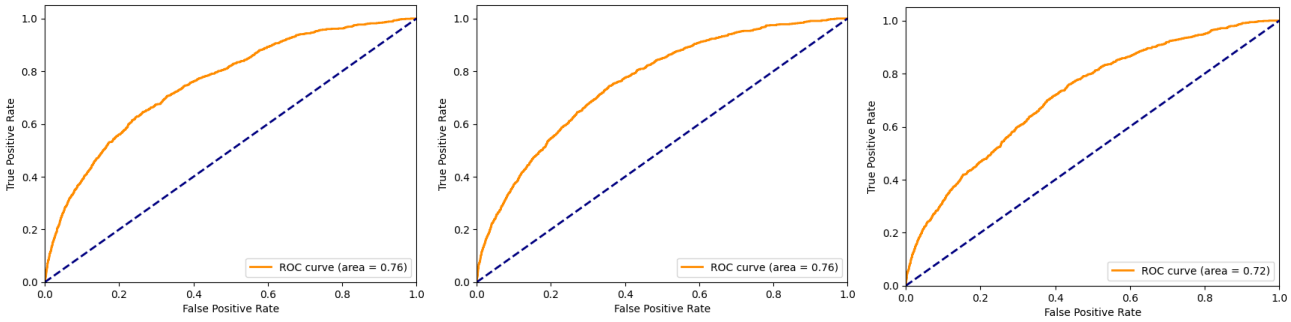
As explained in the previous section, we build a voting classifier over three models which are CatBoost, LightGBM and XGBoost. Figure 4 shows the ROC curves for each of the models and Table 2 shows the each individual model's parameters and performance. Figure 5 shows the ROC curve for the voting classifier with an ***AUC = 0.90***. We observe a significant improvement in the performance with the use of a voting mechanism which can be attributed to the fact that the voting classifier averages the prediction probabilities instead of a simple majority voting.

We submit the voting classifier as the final Kaggle submission which achieves ***stability_metric = 0.584*** fetching us a rank of approximately 500 out of 2800. The top of the leader-board scores *stability_metric = 0.639* and the base model in the starter notebook scores *stability_metric = 0.360*.

4. Process

The results obtained in this project are the culmination of numerous experiments. This section details some of the efforts which were not as successful and did not make it to the final proposed pipeline.

The first challenge faced was to read and organize the dataset. To put into context, the data was present in multiple separate CSV files which had to be combined together for each person based on each person's case ID. Each unique CSV file, has data about the person and their financial history, details about their first circle of people and their financial history and so on. Details of people upto the second circle were provided. This data had to be organized



(a) (b) (c)

Figure 4. ROC curves for (a) CatBoost, (b) LightGBM, and (c) XGBoost.

$$stability_metric = mean(gini) + 88.0.min(0, a) - 0.5.std(residuals) \tag{1}$$

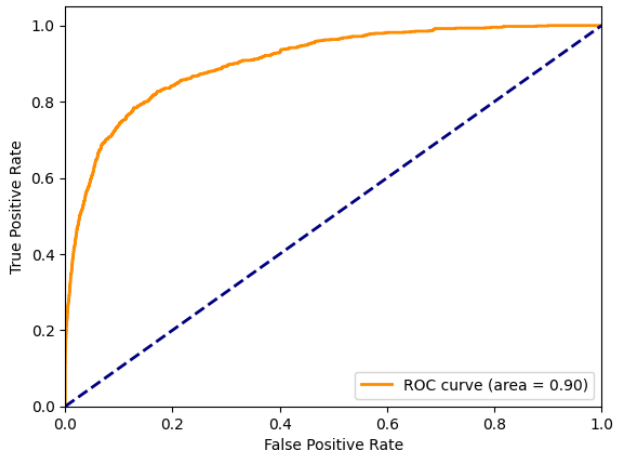


Figure 5. ROC curve for the Voting Classifier.

into meaningful records of an individual. Additionally, since the data size was in the order of GB, regular libraries like pandas were not up to the task to quickly read such files. Instead a library called polars had to be used to read files. The data reading and organization into meaningful dataframes was a crucial step in ensuring all our further operations on the dataset continue smoothly.

Further, once the dataset was organized, the data had a lot of irregularities and missing values which had to be cleaned in order to be used further. This required an experimentation of multiple preprocessing methods to effectively deal with the problems that come with real world data.
On observing the data, it was seen that there were a lot of missing values in many columns. These values needed to be filled with a representative number or be completely removed. Since deleting records which had missing values was not a viable option since almost all of the records had at least 1 missing value, values needed to be imputed. However, upon further investigation, it was seen that 30 features out of the total 480 had less than 10% values in them filled. Filling the missing fields with statistically average values did not make any sense since the sample size was very tiny. Therefore in the final pipeline, these features were discarded as they conveyed very little meaning to the record.
Apart from missing value imputation there were other problems that needed to be addressed before an ML or DL model can be fit on this data. In order to make more meaningful features, some clever feature engineering was used. For example, dates were converted to time durations instead. Since date intervals of a credit line may not make much sense to a model, durations will force the model to make better correlation between other features.

To evaluate the models and pre-processing pipeline trained, repeatedly submitting to Kaggle was not a viable option since each submission takes around 7 hours to evaluate and generate a score. Therefore, an on device evaluation method had to be devised. Therefore, the metric used to evaluate performance was the Area under ROC curve (AUC) which shows how well a model is able to perform in classification scenarios. A value closer to one denotes that the model is able to almost perfectly distinguish between the different classes. A value near 0.5 denotes that the model is only performing as good as a random classifier and has therefore not learnt anything meaningful. A comparison of AUC scores obtained for all the models experimented is presented in Table 1.

Next, as mentioned in the initial project report, regression

Table 2. Model Performance Summary

Model Name	N_estimators	Max Depth	Learning Rate	AUC Score
CatBoost	2000	6	0.03	0.76
LightGBM	2000	10	0.05	0.76
XGBoost	1000	10	0.05	0.72

and classification models were tested on the dataset. Since, there are 10 values that the model needs to predict, the task can be treated as classification. Two classification models were fit on this dataset - SVM, and random forest. Both these models performed worse than their regression counterparts. They got an AUC score of 0.47 and 0.45 respectively. This indicated that regression models would be better suited for the task and therefore the project was solely focused on regression models from then on. This poor performance could be attributed to the fact that the outputs were not treated as risk scores, but instead 10 classes which did not convey as much meaning.

As a last experiment, a lot of effort was spent in getting a deep neural network to work. This came with its own set of challenges as neural networks require all fields to be numeric and not categorical. Traditional encoding methods would not suffice since the labels were not a fixed set and were very high in number. This ruled out one hot encoding and label encoding methods. With those out of the picture, the only option was to manually feature engineer the categories into some sort of numbers or to drop the categorical columns. Due to a lack of time, feature engineering was not possible and dropping all the high dimensional categorical features would cause too much of a lack of data. Hence, after much effort this route was abandoned and the final pipeline consisted of an ensemble model of three boosting algorithms.

5. Contributions

We'd like to thank the author of the posts (Narayanan, 2021a) and (Narayanan, 2021b) for guiding us in the right direction and expediting our progress. Further, we'd like to thank the authors of the python libraries scikit-learn, pandas, polars, xgboost, catboost and lightgbm. Finally, we'd like to express our gratitude to the Home Credit group for hosting the competition and USC and CSCI567 for guiding us through the entire project.

References

et al., A. M. Home credit default risk, 2018. URL https://kaggle.com/competitions/home-credit-default-risk.

Narayanan, D. Home credit default risk (part 1) : Business understanding, data cleaning and eda. *Medium*, 2021a.

Narayanan, D. Home credit default risk (part 3): Modelling — ii and model deployment. *Medium*, 2021b.

Rutecki, M. Voting classifier for better results. 2023.